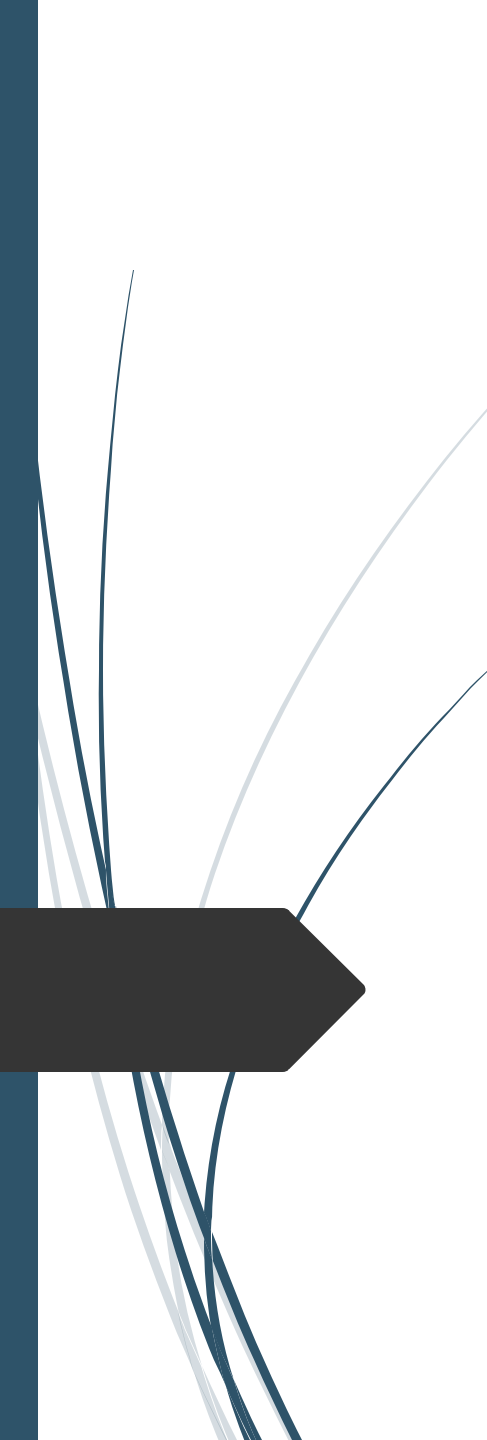




# Basi di Dati - Modulo Laboratorio A.a. 2025-2026

## Lezione 4: Interrogazioni nidificate e insiemistiche

Dr. Beatrice Amico



# Interrogazioni nidificate

SQL permette la costruzione di interrogazioni che presentano al loro interno altre interrogazioni.

La nidificazione può avvenire nelle clausole **SELECT**, **FROM**, **WHERE**.

L'uso più comune è la nidificazione nella clausola **WHERE**.



# Interrogazioni nidificate

## Clausola FROM

Con l'utilizzo di questa opzione, è possibile comporre catene di interrogazioni, in cui ciascuna interrogazione utilizza il risultato della precedente come se fosse una tabella di base.

```
SELECT titolo, prezzoIntero
FROM Mostra, ( SELECT MAX ( prezzoIntero )
FROM Mostra ) AS T( prezzoMax )
WHERE prezzoIntero = prezzoMax;
```



# Interrogazioni nidificate

## **Clausola WHERE**

Il predicato (predicato complesso) nella clausola WHERE confronta un valore (ottenuto come espressione valutata sulla singola riga), con il risultato dell'esecuzione di un'interrogazione SQL (in generale un insieme di valori).

Occorre estendere gli operatori di confronto per poter realizzare la comparazione tra un valore e un insieme di valori.



# Interrogazioni nidificate

Soluzione offerta da SQL: estensione degli operatori di confronto ( $>$ ,  $<$ ,  $=$ ,  $<=$ ,  $>=$ ,  $<>$ ) con ANY/SOME e ALL (IN, NOT IN).

**ANY/SOME**: specifica che la riga soddisfa la condizione se risulta vero il confronto (con l'operatore specificato) tra il valore dell'attributo per la riga e **almeno uno degli elementi** restituiti dall'interrogazione.

**ALL**: specifica che la riga soddisfa la condizione solo se **tutti gli elementi** restituiti dall'interrogazione nidificata rendono vero il confronto.

**IN** e **NOT IN**: usati per rappresentare l'appartenenza o l'esclusione rispetto ad un insieme (identici a  $=$  **ANY** o  $<>$  **ALL**)

# Operatore ANY/SOME

```
expression operator ANY ( subquery )  
expression operator SOME ( subquery )
```

- **expression** è un'espressione che coinvolge attributi della **SELECT** principale.
- **(subquery)** è una **SELECT** che deve restituire UNA sola colonna;
- **operator** è un operatore di confronto, come '=', '>=', ...
- **ANY**: è vero se la condizione è vera per almeno un valore della subquery.
- **SOME** è uno sinonimo di **ANY**.

# Operatore ANY/SOME

**Visualizzare il nome degli insegnamenti che hanno un numero di crediti inferiore alla media dell'ateneo di un qualsiasi anno accademico.**

```
SELECT DISTINCT I.nomeins , IE.crediti
FROM Insegn I JOIN InsErogato IE ON I.id=IE.id_insegn
WHERE IE.crediti < ANY (
  SELECT AVG ( crediti ) FROM InsErogato
  WHERE modulo =0
  GROUP BY annoaccademico
);
```

11607 righe

# Operatore ALL

expression operator ALL( subquery )

- **expression** è un'espressione che coinvolge attributi della **SELECT** principale.
- **(subquery)** è una **SELECT** che deve restituire UNA sola colonna;
- **operator** è un operatore di confronto, come '=', '>=', ...
- **ALL**: La condizione è vera solo se è vera per TUTTI i valori della subquery.

# Operatore ALL

**Trovare il nome degli insegnamenti con almeno un docente e crediti maggiori rispetto ai crediti di ciascun insegnamento del corso di laurea con id=6. (si considerano solo occorrenze insegnamento genitore).**

```
SELECT DISTINCT I. nomeins , IE. crediti
FROM Insegn I JOIN InsErogato IE ON I.id = IE. id_insegn
JOIN Docenza D ON IE.id = D. id_inserogato
WHERE IE. modulo = 0 AND IE. crediti > ALL (
  SELECT crediti FROM InsErogato
  WHERE id_corsostudi = 6 AND modulo =0
);
```

(1485 righe )



# Operatore IN

E' un operatore utilizzabile nei predicati complessi.

```
[ROW] ( expr [ , ... ] ) IN ( subquery )
```

- **expr** è un'espressione costruita con un attributo della query principale. Ci possono essere una o più espressioni.
- La (**subquery**) deve restituire un numero di colonne pari al numero di espressioni in (**expr** [...]).
- I valori dell'espressioni vengono confrontati con i valori di ciascuna riga del risultato di (subquery).
- Il confronto ritorna vero se i valori sono uguali ai valori di almeno una riga della subquery.



# Operatore IN

**Trovare gli impiegati che hanno stesso nome e cognome di qualcuno in un'altra azienda.**

```
SELECT l.nome , l.cognome  
FROM Impiegato l  
WHERE ROW( l.nome , l.cognome ) IN (  
SELECT l1.nome, l1. cognome FROM ImpiegatoAltraAzienda l1  
);
```

# Operatore EXISTS

E' una clausola utilizzabile nei predicati complessi.

`EXISTS (subquery)`

- (subquery) è una `SELECT`.
- `EXISTS` ritorna falso se (subquery) non contiene righe; altrimenti vero.
- `EXISTS` è significativo quando nella (subquery) si selezionano righe usando i valori della riga corrente nella `SELECT` principale: **data binding**. Vale a dire se subquery è un'interrogazione dipendente dalla query esterna.

# Operatore EXISTS

**Determinare i nomi degli impiegati che sono diversi tra loro ma di pari lunghezza.**

```
SELECT I. nome
FROM Impiegato I
WHERE EXISTS (
  SELECT 1 FROM Impiegato I1 WHERE I.nome <> I1. nome
  AND CHAR_LENGTH( I.nome ) = CHAR_LENGTH( I1. nome )
);
```

I.nome nella subquery è il valore di nome nella riga corrente della SELECT principale.

# Operatore EXISTS

Se si usano attributi esterni nella subquery (data binding), la subquery deve essere valutata per ogni riga della SELECT principale.

**Visualizzare il nome e il cognome dei docenti, escludendo i coordinatori, che hanno tenuto almeno due insegnamenti (o moduli) con più di 24 crediti nel 2010/2011.**

```
SELECT P.nome , P.cognome
FROM Persona P
WHERE EXISTS (
  SELECT 1 FROM Docenza D JOIN InsErogato IE ON D. id_inserogato = IE.id
  WHERE IE. crediti > 24 AND D. coordinatore = '0'
  AND IE. annoaccademico = '2010/2011' AND D. id_persona = P.id
  GROUP BY D. id_persona
  HAVING COUNT (*) >=2
);
```

# Operatore EXISTS

**Visualizzare il nome dei corsi di studio che nel 2006/2007 non hanno erogato insegnamenti il cui nome contiene la sottostringa 'Info'.**

```
SELECT CS. nome FROM CorsoStudi CS
WHERE NOT EXISTS (
  SELECT I FROM InsErogato IE JOIN Insegn I ON IE. id_insegn =
  I.id
  WHERE I.nomeins LIKE '%Info%'
  AND IE.annoaccademico = '2006/2007'
  AND IE.id_corsostudi = CS.id);
```

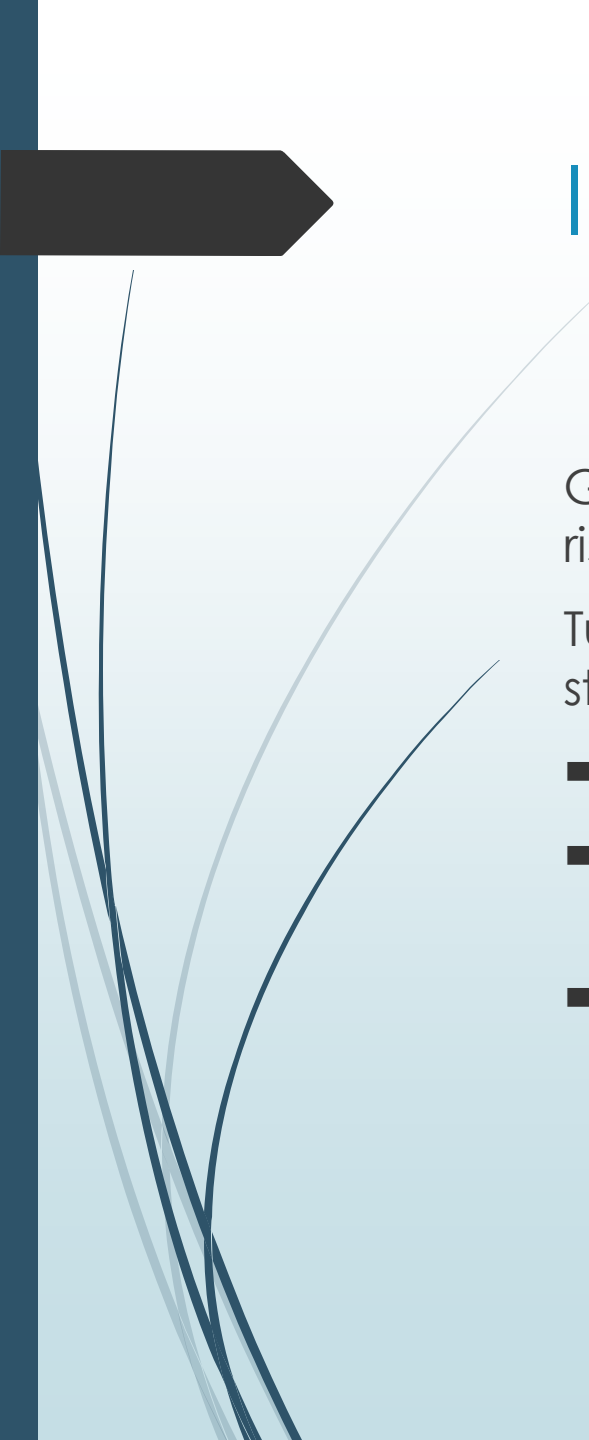
# Interrogazioni di tipo insiemistico

```
query_1 { UNION | INTERSECT | EXCEPT } [ALL] query_2
```

SQL mette a disposizione anche degli **operatori insiemistici UNION, INTERSECT e EXCEPT** per manipolare i risultati di più query.

Gli operatori insiemistici si possono utilizzare solo al livello più esterno di una query, operando sul risultato di due o più clausole SELECT.

Si possono avere sequenze di UNION/INTERSECT/EXCEPT



# Interrogazioni di tipo insiemistico

Gli operatori si possono applicare solo quando **query1** e **query2** producono risultati con lo stesso numero di colonne e di tipo compatibile fra loro.

Tutti gli operatori eliminano i **duplicati** dal risultato a meno che **ALL** non sia stato specificato.

- **UNION** aggiunge il risultato di **query2** a quello di **query1**.
- **INTERSECT** restituisce le righe che sono presenti sia nel risultato di **query1** sia in quello di **query2**.
- **EXCEPT** restituisce le righe di **query1** che non sono presenti nel risultato di **query2**. In pratica esegue la differenza insiemistica.

# Interrogazioni di tipo insiemistico

Visualizzare i nomi degli insegnamenti e i nomi dei corsi di laurea che non iniziano per 'A' mantenendo i duplicati.

```
SELECT nomeins  
FROM Insegn  
WHERE NOT nomeins LIKE 'A%'
```

```
UNION ALL
```

```
SELECT nome  
FROM CorsoStudi  
WHERE NOT nome LIKE 'A%';
```

(8253 righe)



# Interrogazioni di tipo insiemistico

Visualizzare i nomi degli insegnamenti che sono anche nomi di corsi di laurea.

```
SELECT nomeins  
FROM Insegn
```

```
INTERSECT ALL
```

```
SELECT nome  
FROM CorsoStudi;
```

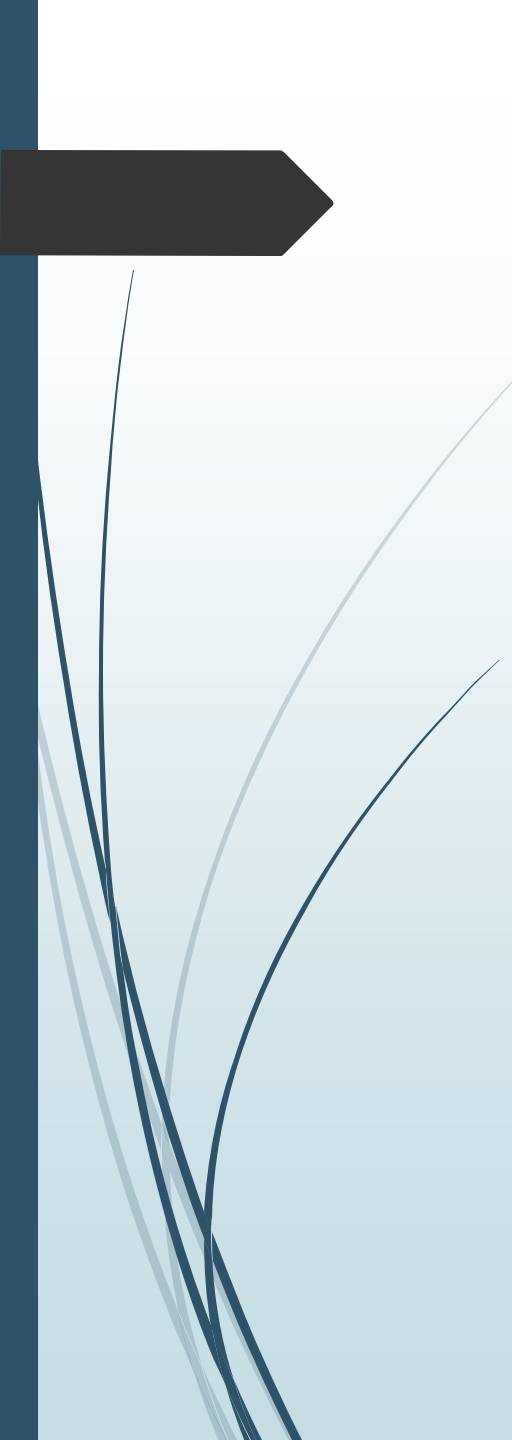


# Interrogazioni di tipo insiemistico

Visualizzare i nomi degli insegnamenti che NON sono anche nomi di corsi di laurea.

```
SELECT nomeins  
FROM Insegn  
  
EXCEPT  
  
SELECT nome  
FROM CorsoStudi ;
```

(7709 righe)



# Le VISTE

Le viste sono tabelle virtuali il cui contenuto dipende dal contenuto delle altre tabelle della base di dati.

In SQL le viste vengono definite associando un nome ed una lista di attributi al risultato dell'esecuzione di un'interrogazione.

Ogni volta che si usa una vista, si esegue la query che la definisce.

Nell'interrogazione che definisce la vista possono comparire anche altre viste.

SQL **non ammette** però:

- dipendenze immediate (definire una vista in termini di se stessa) o ricorsive (definire una interrogazione di base e una interrogazione ricorsiva);
- dipendenze transitive circolari (V1 definita usando V2, V2 usando V3, . . . , Vn usando V1).

# Le VISTE

```
CREATE [ TEMP ] VIEW nome [( col_name [, ...]) ] AS query
```

**TEMP**: la vista è temporanea. Quando ci si sconnette, la vista viene distrutta. È un'estensione di PostgreSQL. Nella base di dati did2014 si possono fare solo viste temporanee.

- **column\_name**: nomi delle colonne che compongono la vista; se non vengono specificati, sono ricavati dai nomi delle colonne restituiti dalla query, eventualmente tramite alias.
- **query** deve restituire un insieme di attributi pari e nel medesimo ordine a quello specificato con ( **column\_name** [, ...] ) se presente.
- SQL permette che una vista sia aggiornabile solo quando ciascuna tupla della vista corrisponde a una sola tupla di ciascuna tabella di base.



# Le VISTE

**Definire la vista che contiene gli insegnamenti erogati completi di nomeins e codiceins presi dalla tabella Insegn.**

```
CREATE TEMP VIEW InsErogatiCompleti AS  
SELECT I. nomeins , I. codiceins , IE .*  
FROM InsErogato IE  
JOIN Insegn I ON IE. id_insegn = I.id;
```

# Le VISTE

**Si vuole determinare quali sono i corsi di studi con il massimo numero di nome di insegnamenti diversi.**

Prima si crea una vista:

```
CREATE TEMP VIEW InsCorsoStudi (Nome , NumIns) AS
SELECT CS.nome , COUNT ( DISTINCT I. nomeins )
FROM CorsoStudi CS
JOIN InsErogato IE ON CS.id=IE. id_corsostudi
JOIN Insegn I ON IE. id_insegn = I.id
GROUP BY CS. nome;
```



# Le VISTE

Poi la si usa:

```
SELECT Nome , NumIns
FROM InsCorsoStudi
WHERE NumIns = ANY (
SELECT MAX ( NumIns ) FROM InsCorsoStudi
);
```

Laurea specialistica in Medicina e Chirurgia: 320